# Starter

1. What is stored in a stack frame?
2. How/when does the call stack grow and shrink?

# Starter

1. What is stored in a stack frame?
2. How/when does the call stack grow and shrink?

1. Local Variables, parameters, return address
2. When a method is called the calls tack grows by one stack frame.
   When the method ends, the stack shrinks as the frame is removed.

# Lists 2

Keywords: Array; List; Data structure; Dynamic memory allocation; Object; New;

# British Informatics Olympiad

A fun, but very challenging programming competition. Can be completed in any programming language.
3 hours, 3 problems.

Each letter in the alphabet can be given a value based on its position in the alphabet, A being 1 through to Z being 26. Two letters can produce a third letter by adding together their values, deducting 26 from the sum if it is greater than 26, and finding the letter whose value equals this result.

A simple encryption scheme for a string is as follows. Working from left to right, after leaving the leftmost character alone, each character is replaced with the character produced by adding it to the character to its immediate left.

For example: The string ENCRYPT goes through the steps ENCRYPT → ESCRYPT → ESVRYPT → ESVNYPT → ESVNMPT -> ESVNMCT -> ESVNMCW

To participate, you must have completed at least one question from last year's paper. More information on GClassroom, or email Isabel (imc@barton.ac.uk)

# In pairs, discuss

## What is the difference between an array and a list

Write down ideas on a whiteboard:

- How are they declared and accessed in C#?
- What properties do they have?
- How are they stored in memory?

# In pairs, discuss

## What is the difference between an static and dynamic memory allocation?

Write down ideas on a whiteboard:

- Where in the process's memory are they stored?
- Why might dynamic be advantageous?
- What are the overheads of dynamic?

# Static

**Static**: we will have values of a fixed size such as `int` and `float`, which both always need 32 bits of memory.

These are **PRIMITIVE DATA TYPES**

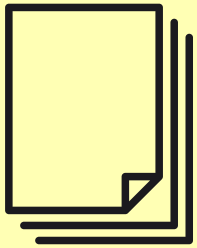
These are stored in the **STACK**

# Static

**Static**: we will have values of a fixed size such as `int` and `float`, which both always need 32 bits of memory.
These are **PRIMITIVE DATA TYPES**
These are stored in the **STACK**

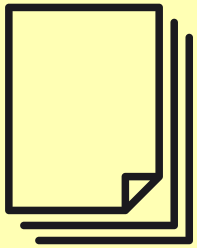
Can you think of any more?

# Dynamic



**Dynamic**: we may also have some values where
we <u>don't know exactly how much memory space</u> we will need at all times, such as `List`.
These are called **OBJECTS**
These are stored in the **HEAP**

# Dynamic



**Dynamic**: we may also have some values where
we <u>don't know exactly how much memory space</u> we will need at all times, such as `List`.
These are called **OBJECTS**
These are stored in the **HEAP**

Can you think of any more?

# Storing variables of a fixed length

When we store a variable of a fixed length, we can simply put it into a memory location in the stack:

| M.A. | Data |
|---|---|
| 0001 | |
| 0002 | |
| 0003 | |
| 0004 | |

# Storing variables of a fixed length

When we store a variable of a fixed length, we can simply put it into a memory location in the stack:

| M.A. | Data |
|---|---|
| 0001 | 0000 0000 0000 0001<br>1101 0000 0100 0101 |
| 0002 | |
| 0003 | |
| 0004 | |

```
int myNum = 118853
```

# Storing variables of a fixed length

When we store a variable of a fixed length, we can simply put it into a memory location in the stack:

| M.A. | Data |
|---|---|
| 0001 | 0000 0000 0000 0001<br>1101 0000 0100 0101 |
| 0002 | 0000 0000 0000 0000<br>0000 0000 0110 0011 |
| 0003 | |
| 0004 | |

```
char myLetter = 'c'
```

# Storing variables of a fixed length

When we store a variable of a fixed length, we can simply put it into a memory location in the stack:

| M.A. | Data |
|---|---|
| 0001 | 0000 0000 0000 0001<br>1101 0000 0100 0101 |
| 0002 | 0000 0000 0000 0000<br>0000 0000 0110 0011 |
| 0003 | 0011 1111 1110 0001<br>1101 0000 0111 1100 |
| 0004 | 1000 0100 1011 0101<br>1101 1100 1100 0110 |

```
double myNum = 0.5567
```

# Storing Objects

Storing objects is more complicated, as we don't know how much space they will need.

# Storing Objects

Storing objects is more complicated, as we don't know how much space they will need.
But to store a variable, we **must** place something in the stack.

# Storing Objects

Storing objects is more complicated, as we don't know how much space they will need.
But to store a variable, we **must** place something in the stack.
We use an **object reference** (a pointer) which points to a memory location in the heap.

# Storing Objects

Storing objects is more complicated, as we don't know how much space they will need.
But to store a variable, we **must** place something in the stack.
We use an **object reference** (a pointer) which points to a memory location in the heap.

| | |
|---|---|
| 0001 | |
| 0002 | |
| 0003 | |
| 0004 | |

# Storing Objects

Storing objects is more complicated, as we don't know how much space they will need.
But to store a variable, we **must** place something in the stack.
We use an **object reference** (a pointer) which points to a memory location in the heap.

| | |
|---|---|
| 0001 | |
| 0002 | |
| 0003 | |
| 0004 | |

```
List<int> myList = new List<int>();
```

# Storing Objects

Storing objects is more complicated, as we don't know how much space they will need.
But to store a variable, we **must** place something in the stack.
We use an **object reference** (a pointer) which points to a memory location in the heap.

| | |
|---|---|
| 0001 | |
| 0002 | |
| 0003 | |
| 0004 | |

**HEAP**:

| | |
|---|---|
| Value | null |
| Next | null |

```
List<int> myList = new List<int>();
```

# Storing Objects

Storing objects is more complicated, as we don't know how much space they will need.
But to store a variable, we **must** place something in the stack.
We use an **object reference** (a pointer) which points to a memory location in the heap.

| | |
|---|---|
| 0001 | 0x0010 5F4C |
| 0002 | |
| 0003 | |
| 0004 | |

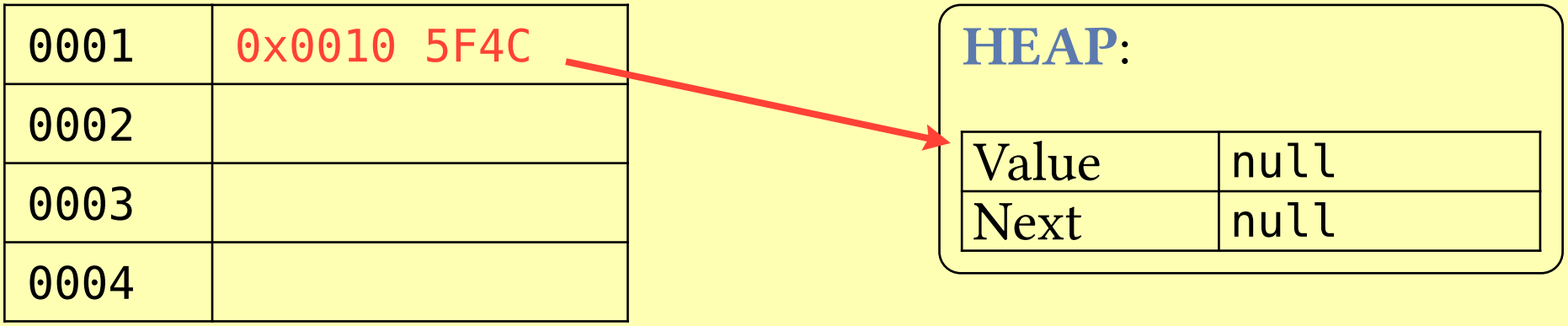
**HEAP**:

| | |
|---|---|
| Value | null |
| Next | null |

```
List<int> myList = new List<int>();
```

# Storing Objects

Storing objects is more complicated, as we don't know how much space they will need.
But to store a variable, we **must** place something in the stack.
We use an **object reference** (a pointer) which points to a memory location in the heap.




```
List<int> myList = new List<int>();
```

# Storing Objects

Every time we add another item to the list, we update Value and Next.

**HEAD**:

| | |
|---|---|
| Value | `null` |
| Next | `null` |

| | |
|---|---|
| Value | `null` |
| Next | |

| | |
|---|---|
| Value | `null` |
| Next | `null` |

| | |
|---|---|
| Value | `null` |
| Next | `null` |

# Storing Objects

Every time we add another item to the list, we update Value and Next.

**HEAD**:

| | |
|---|---|
| Value | 4 |
| Next | `null` |

| | |
|---|---|
| Value | `null` |
| Next | |

| | |
|---|---|
| Value | `null` |
| Next | `null` |

| | |
|---|---|
| Value | `null` |
| Next | `null` |

```
myList.Add(4);
```

# Storing Objects

Every time we add another item to the list, we update Value and Next.

**HEAD**:

| | |
|---|---|
| Value | `4` |
| Next | `null` |

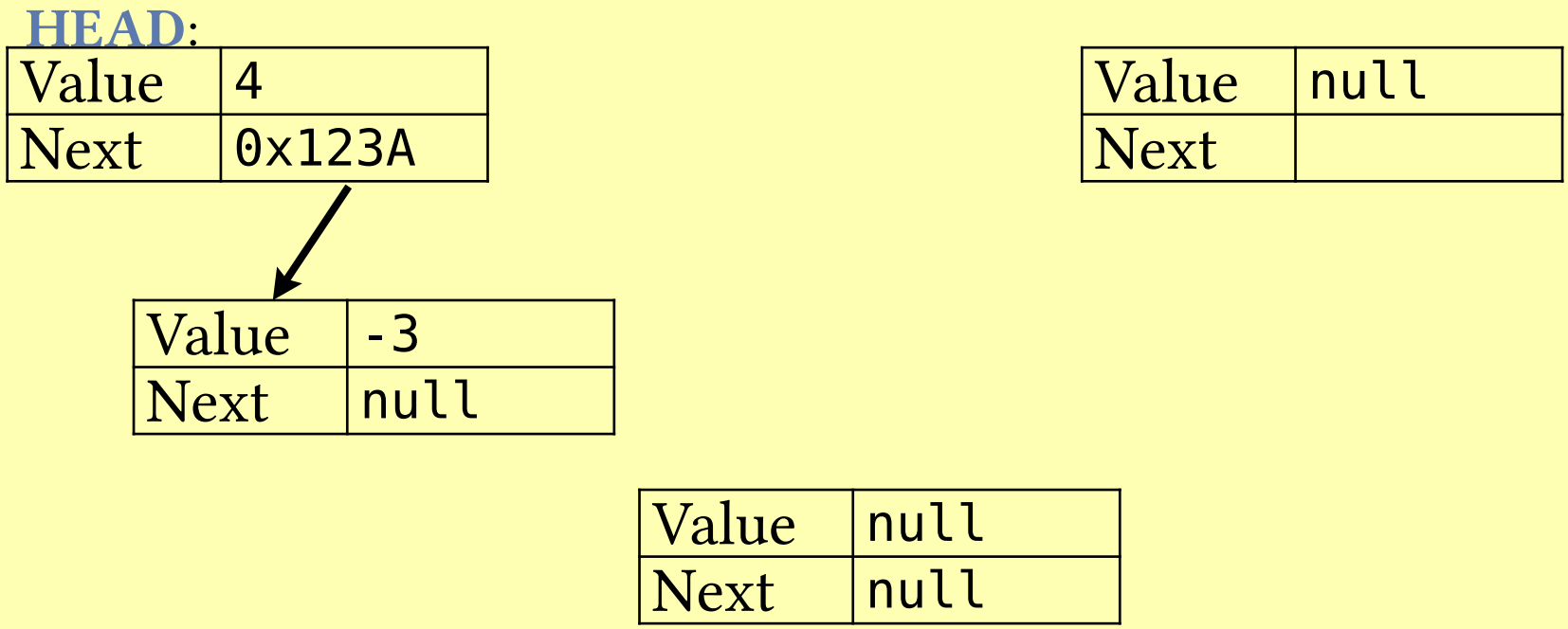
| | |
|---|---|
| Value | `null` |
| Next | |

| | |
|---|---|
| Value | `null` |
| Next | `null` |

| | |
|---|---|
| Value | `null` |
| Next | `null` |

# Storing Objects

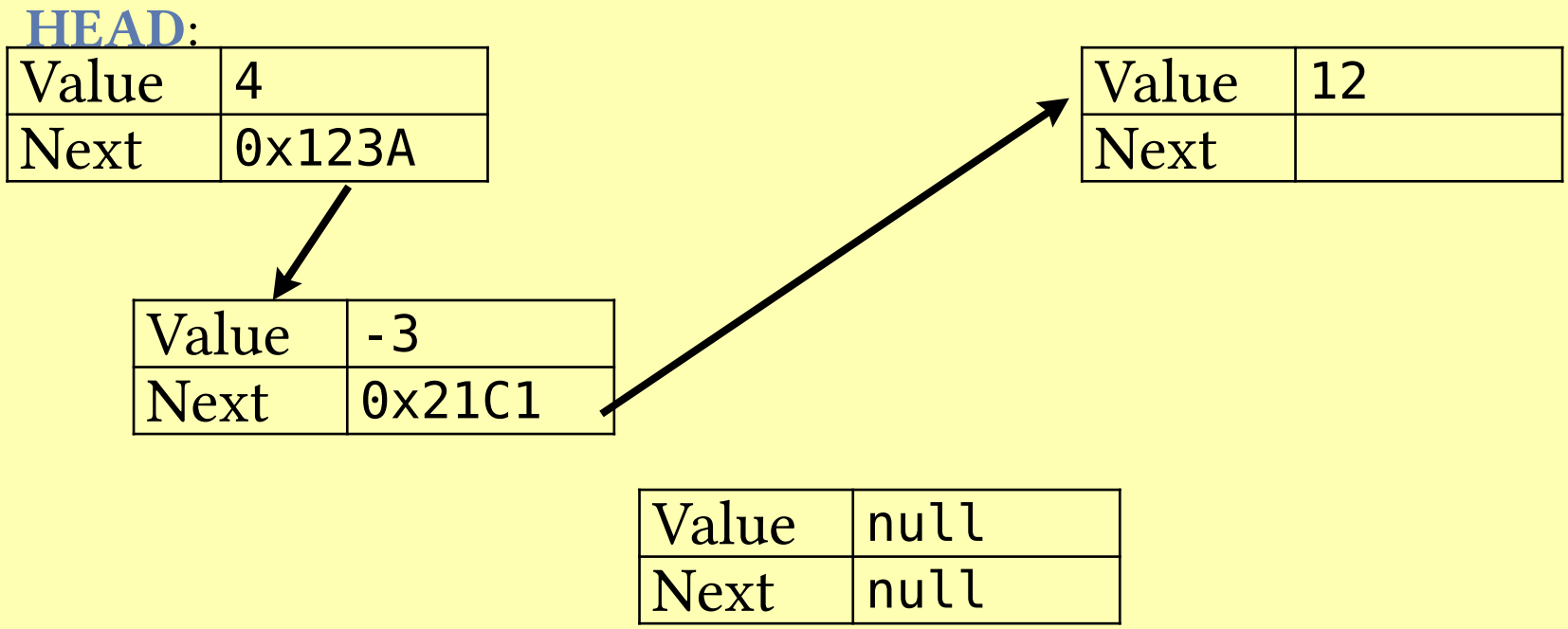
Every time we add another item to the list, we update Value and Next.




```
myList.Add(-3);
```

# Storing Objects

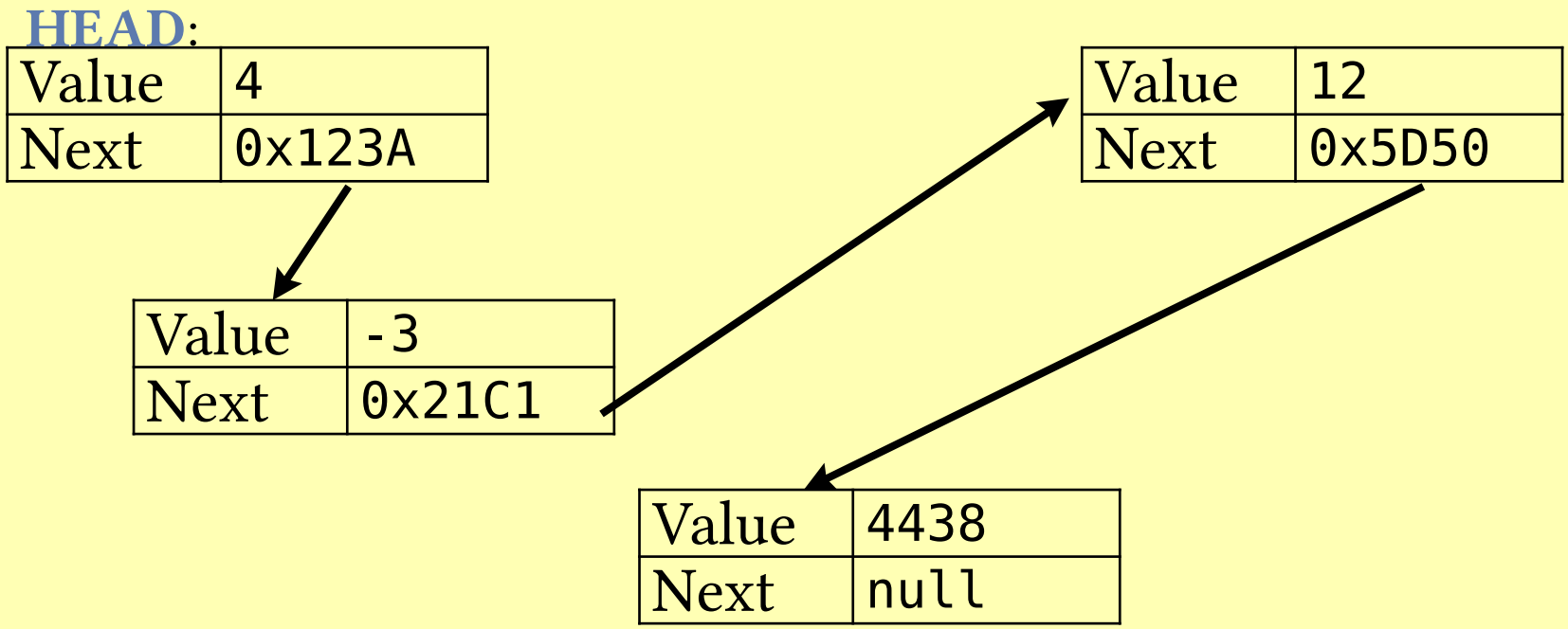
Every time we add another item to the list, we update Value and Next.




```
myList.Add(12);
```

# Storing Objects

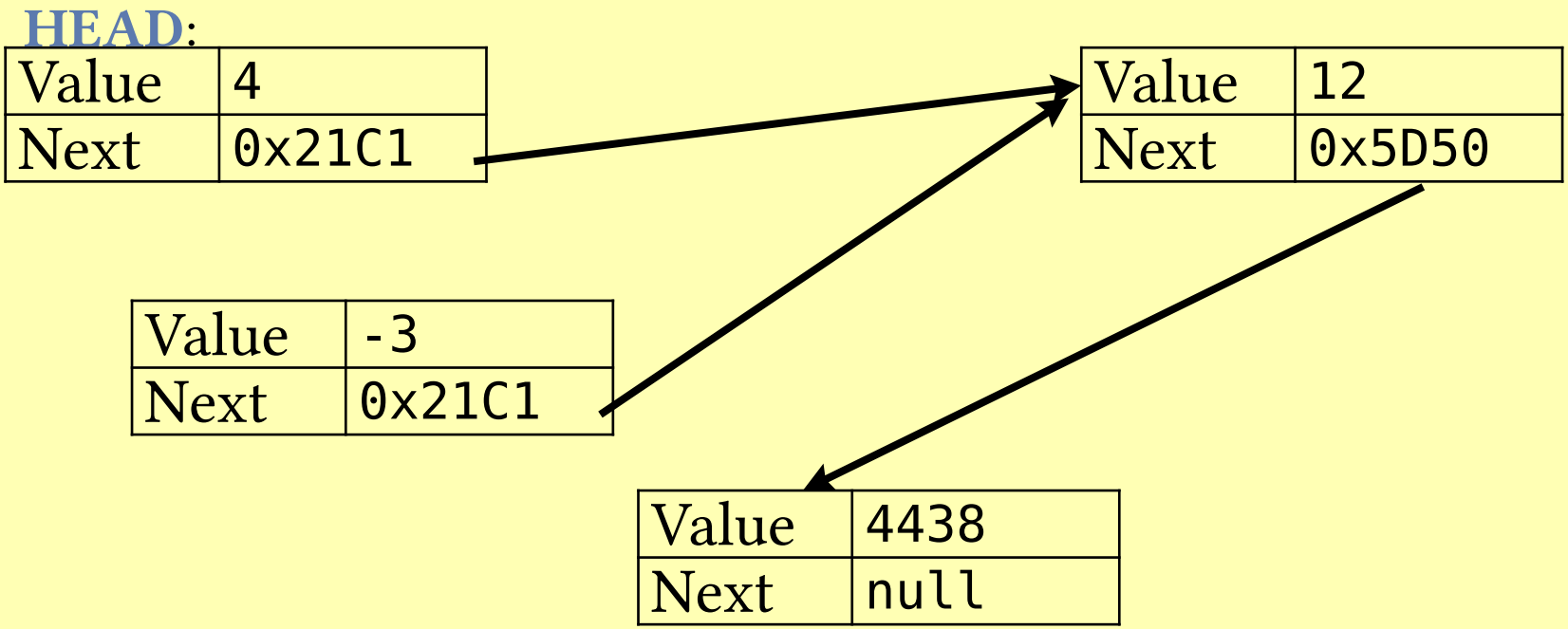
Every time we add another item to the list, we update Value and Next.




```
myList.Add(4438);
```

# Storing Objects

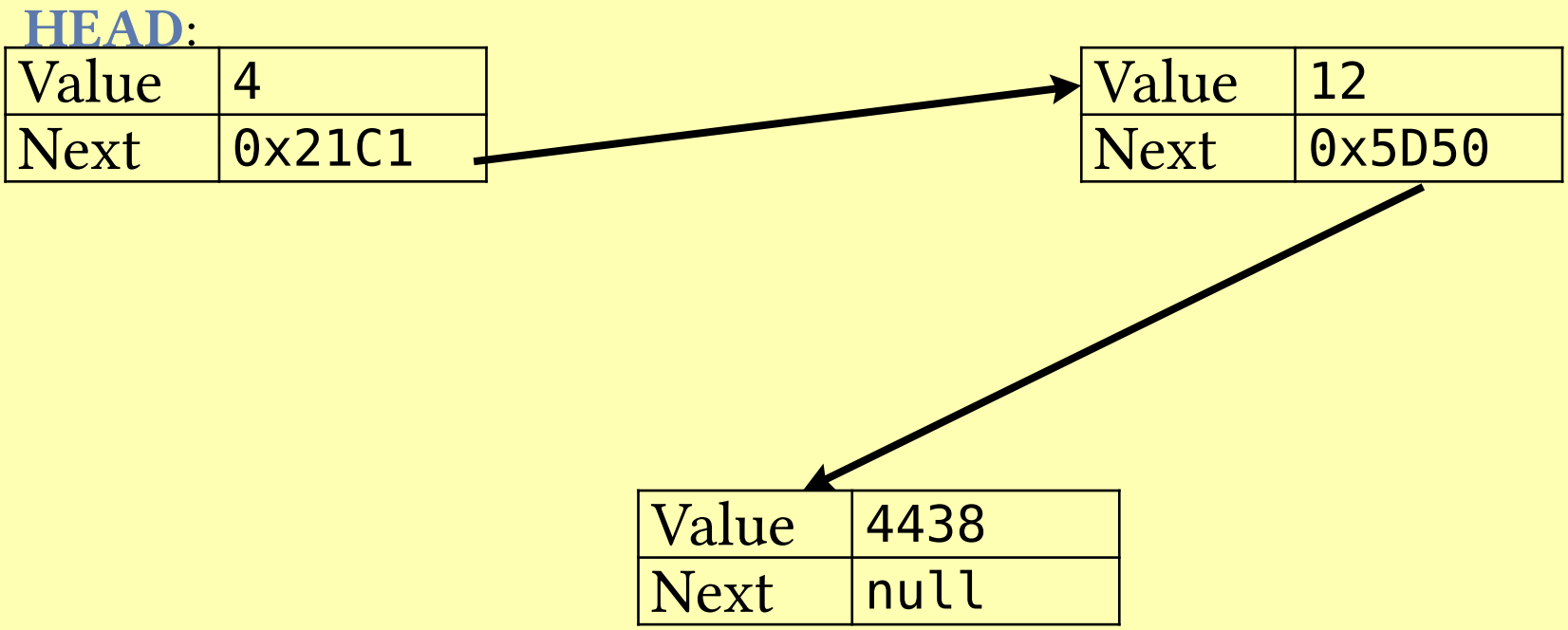
Every time we add another item to the list, we update Value and Next.




```
myList.Remove(-3);
```

# Storing Objects

Every time we add another item to the list, we update Value and Next.




The cleanup of this element is handled by the Operating System.

# Your Turn

On BPCompSci: **W109 - C# Lists**

Example Code:

```
for(int i = 0; i<numbers.Count -1; i++)
{
     Console.WriteLine(numbers[i]);
}
foreach(int num in numbers)
{
      Console.WriteLine(num);
}
```

```
List<int> numbers;
numbers = new List<int>();

numbers.Add(6);
numbers.Add(-3);
numbers.Add(4);

numbers.Remove(-3);

Console.WriteLine(numbers.Count);

Console.WriteLine(numbers[0] +
                  " " + numbers[1]);

numbers = new List<int>();

Console.WriteLine(numbers.Count);
```